ractions, we can better validate requirements, dogfood existing functionality, and contribute security capabilities through a cohesive, cross-functional partnership.

SECTION: security/product-security/security-platforms-architecture/security-interlock/customer-zero-triage-process.md

New Customer Zero Requests

When Customer Zero requests are created using our request templates, they are automatically labeled with C0Workflow::Backlog, which makes them visible on this issue board (internal only).

Triagers may choose to subscribe to the C0Workflow::Backlog label and/or periodically monitor the board to identify new issues that need to be triaged.

How to Triage New Issues

1. Select an issue that does NOT start with the prefixes DRAFT: or WIP:. Those issues are ones that submitters have started, but they still need to provide additional information before we review them. That's fine! We'd prefer to have visibility into what's coming anyway.
1. Apply the label C0Workflow::Initial Triage to indicate that someone is already triaging this issue so no one duplicates your efforts. You can do this manually or by moving the Issue to the next column on the issue board (internal only). If your triage takes more than just a few minutes, assign the issue to yourself so this shows up in your TODOs and you don't lose track.
1. Review the information provided in the Issue and determine if sufficient information has been provided for someone to work it. For example, are all of the key fields populated? Do you understand the feature and have all of the information you'd need to do a review?
   - If Not: Tag the requestor and key contacts in a comment and ask them clarifying questions. Apply the label C0::Waiting on Requestor to the Issue manually or by moving it to the next column on the issue board (internal only). Ask that they tag you directly when they've provided the additional information so you can re-review and follow the next steps.
1. Once you have determined there is sufficient information in the Issue, first set the milestone appropriately, given both the Timeline Requirements in the Issue and our SLOs, which state requests received at least 10 calendar days before the next milestone starts should have feedback provided within the next milestone. Requests received later than that will typically be prioritized for milestone+2.
1. Determine which team(s) are best suited to provide the appropriate feedback. If you are unsure which teams' feedback is required, ask in product-security-department-only. Once you have determined the appropriate team(s), take the following steps:

Team	Steps
AppSec	1. Apply the Application Security Team label 2. Tag @gitlab-com/gl-security/product-security/appsec and ask them to incorporate into their milestone planning
PSIRT	Tag @gitlab-com/gl-security/product-security/appsec/psirt-group and ask them to incorporate into their milestone planning
Vuln Management	Tag @gitlab-com/gl-security/product-security/vulnerability-management and ask them to incorporate into their milestone planning
Not Listed	Tag team members individually, and update this Handbook page if you receive

alternate instructions |

1. Apply the label C0Workflow::On Deck to indicate this has been triaged and assigned.

You're Done!

SECTION: security/product-security/security-platforms-architecture/security-interlock/internal-co-create.md

Summary

GitLab has a highly technical workforce outside of the Product/Engineering divisions who are looking to make direct product contributions. This document details a process on how to do this work so we can deliver results for customers.

Process

Project Phases

1. Identify Core Team and Objective

At a minimum, require a Product Manager, Engineering Manager or Tech Lead, and a Receiving Team(s)

Creators
PI = Product/ Project Initiator (Within the Product division)
Collaborate with non Prod/Eng division counterparts
Provides the objectives to be met
Owns the feature vision and requirements
Has final sign-off authority on completed feature
EM = Engineering Manager/ Tech Lead (This will most likely include a Project/Product manager outside the Prod/Eng division)
Responsible for building the feature according to PI and RT requirements
Manages the development lifecycle (design through handoff)
Receivers
RT = Receiving Team(s) (Within the Engineering division)
Vets handoff information for readability and completeness
Will ultimately own the feature long-term with the RPM
Provides input during development
Takes over engineering ownership after handoff
RPM = Receiving Product/ Project Manager (Within the Product division)
May be the same as the PI
Will ultimately own the feature long-term with the RT
Provides input during development
Takes over product/ project ownership after handoff

2. Project Scope and Objectives

PI defines the goal behavior and desired business outcomes
PI + EM identify what·s in and out of scope
EM shares the implementation plan and design proposal for the RT
EM provides timelines for development and handoff negotiated
RT, RPM, EM+PI document review/approval by leadership of both teams on maintenance/featurecategory association
PI + EM define key deliverables, objectives, and dependencies
Ends with documentation of project goal(s) with sign-off from EM, PI, and RT

3. Develop the Project Plan

PI + EM work together to document the use cases and workflows
EM decompose functionality into Milestones, Epics, and Issues
PI + EM work together to coordinate dependencies across orgs (e.g., APIs, data access, infrastructure)
EM documents the test plan, rollout, and deployment
EM estimate resources, budget, and timelines when required
EM plan for Risks and provides mitigation strategies (Security Review)
Ends with Milestones populated with the Issues/ Epics that are part of the Internal Co-Create initiative

4. Execute the Project Plan

EM reports project statuses and blockers according to standards set for all involved stakeholders
EM works with RT + RPM to allocate capacity
PI + EM + RT + RPM attend demos at end of each Milestone
PI + EM provide timely async feedback weekly
EM deploys the deliverables internally and then to external users
  New Services follow the documentation for Production Readiness Review
  Monolith Services follow standard MR procedures
Ends with the code deployed to Live

5. Delivery and Handoff

EM complete any Feature Flags and obvious Tech Debt prior to handover
EM captures lessons learned from the project
EM provide documentation for seamless and easy handoff to RT + RPM
RT + RPM gains competency of the deliverables before accepting ownership
  Documentation
  Knowledge Transfer Syncs
  Recordings of workflow and testing
  Lunch-and-Learn Sessions
  Demos
RT + RPM responsible for monitoring and iterating on deliverables after acceptance
Ends with RT + RPM signing off on receipt of the deliverables

---
SECTION: security/product-security/security-platforms-architecture/security-

interlock/request-customer-zero-validation.md

Customer Zero Overview

Customer Zero Validation is the process where the Product Security Team acts as the first customer for new security features. Our hypothesis is simple: if we can use GitLab's security features effectively, then many of our customers can too.

The Issue Board tracking these requests can be found here.

Request Types

Product Managers or Engineering Teams can engage with the Product Security Team during different phases of feature development through four distinct request types. To streamline our collaboration, we've created specific Issue Templates for each one that help you provide the initial information we need to begin our evaluations.

Idea/Priority Validation

Purpose: To determine if your proposed feature solves a genuine problem for the Product Security Team, and understand its relative priority.

When to use: Early in your product planning cycle, when you have a concept or idea but need to validate whether it addresses a real security need.

Best practices:

- Submit before investing significant resources in requirements development
- Include a clear problem statement that articulates the pain point
- Be open to feedback that might redirect or reshape your initial concept
- Use our feedback to inform your product roadmap priorities

Click here to submit this type of request.

Requirements Gathering

Purpose: To collect detailed requirements from the Product Security Team about what would make the feature valuable for our daily work.

When to use: After validating that your feature idea is worth pursuing, but before designing a specific solution.

Best practices:

- Submit after idea validation but before solution design
- Provide clear context about the problem you're solving
- Be specific about the intended user and use cases
- Use our feedback to ensure your solution design addresses our needs

Click here to submit this type of request.

Solution Validation

Purpose: To verify that your proposed design and implementation will address the Product Security Team's requirements and integrate effectively into our workflows.

When to use: After designing your solution approach but before beginning implementation.

Best practices:

- Include mockups, wireframes, or detailed descriptions of the user experience
- Use our feedback to refine your solution before development begins

Click here to submit this type of request.

Internal Testing

Purpose: To test the feature in real-world scenarios and identify any functionality, usability, or integration issues before release.

When to use: After implementation but before public release, when the feature is functional enough for realistic testing.

Best practices:

- Ensure the feature is sufficiently developed for meaningful testing
- Provide clear setup instructions and testing scenarios
- Allow sufficient time to incorporate feedback before scheduled release
- Use our feedback to make final refinements and determine release readiness

Click here to submit this type of request.

Timeline Expectations

The Product Security Team is excited to partner with you to make GitLab's security features the best they can be! While we strive to accommodate all requests as quickly as possible, our response timelines follow these Service Level Objectives (SLOs):

- Requests received at least 10 calendar days before the next milestone starts: We'll provide feedback within the next milestone
- Requests received later than 10 days before the next milestone starts: Our response will typically be provided in milestone+2

This advance notice helps us allocate capacity to provide you with the most thorough and valuable feedback. For critical features, consider engaging with us early and through multiple request types for the best collaborative results.

Remember

Our hypothesis is that if the Product Security Team can successfully use GitLab's security features, then our customers can too. Customer Zero Validation is how we test that hypothesis

and deliver complete, valuable security capabilities from day one.

Frequently Asked Questions

Q: If we'd like to involve the Product Security Team during every phase of feature development, should we open one issue for the feature or one issue per request type?

A: We'd want and need one issue per request type. You're welcome to link them, but the reason is we expect that plans will change throughout the product development lifecycle. We need to ensure we're operating on current information. Additionally, the level of detail should increase as time goes on (i.e., setup instructions will be known for internal testing, but rarely before).

SECTION: security/product-security/security-platforms-architecture/security-research/_index.md

Team Focus

The Security Research team contributes to the Security Vision and Mission through projects that focus on identifying, quantifying, and developing solutions for complex security risks facing GitLab and its users. This work aims to improve the security posture of the product and the company, but always with an eye for contributing new functionality as a differentiator. Additionally, we aim to share our results widely in order to educate and bring awareness to the GitLab Security program.

In order to realize this vision, team projects generally align with the following categories in accordance with the role description:

- Provide security insight
- Development of new security capabilities
- Education

Provide Security Insight

In order to secure GitLab the product and GitLab the company, security insight projects aim to identify, quantify, and communicate technical security risks.

Examples include:

- Security testing of FOSS applications and dependencies used within GitLab
- Introduction and practice of threat modeling
- Publishing of internal technical risk reports

Security Capability Development

Projects in the security capability development category aim to provide novel tools to enable others to innovate securely.

Examples include:

- Package Hunter
- Untamper My Lockfile
- Token leak tooling

Education

We want to ensure that our ideas are available for others to learn from and build upon. To achieve that goal we share our results as widely as appropriate. Additionally, dedicated education projects seek to develop engaging security content to help raise awareness of security concepts.

Examples include:

- Blog posts
- GitLab Security Tech Notes
- Conference presentations
- Writing of security standards
- GitLab documentation updates
- Development of internal training

Even if the outcome of the research is not yielding the expected outcomes the approaches and procedures conducted during the research phase should be documented and published.

Project Lifecycle

The Security Research project lifecycle is designed to quickly iterate through the technical validation of research questions. The technical validation is followed by a business validation in which stakeholders are formally identified and concrete deliverables are defined. This workflow allows for the exploration of new ideas, but ensures that projects are aligned with business objectives. Details on how projects are prioritized are covered in Project Prioritization.

Participants

Team Members

As experts, security research team members are given space to direct their own projects. To do so, they use data from a number of different sources to decide where their focus can be impactful to the business, and the larger security community.

Some sources of data include:

- Their own security background
- Security questions/problems raised within GitLab
- Outreach with GitLab team members

- Security industry trends

Team Manager

The role of the team manager is to support and guide team members in their project. This includes sharing information that they may collect and facilitating communication with other team members. One example of where the manager can be impactful is raising security questions/problems identified within the company.

Stakeholders

In order to ensure that projects are well aligned with business objectives we seek to identify stakeholders within the company to act as sponsors for our projects. A stakeholder can be identified at any time, but we also seek to identify stakeholders once a technical idea has been validated. A team member can volunteer to be a stakeholder by reaching out to the team or opening a Lightbulb issue.

Lightbulb Ideas

Lightbulb ideas are the first step of a security research project. These are inspired research questions
that we believe are worth investigating and are boardly aligned with business objectives. Lightbulb ideas are labeled with ~security-research::lightbulb, and open lightbulb ideas are discussed during the security research sync to help shape our team roadmap. Lightbulb ideas can be opened by anyone interested in partnering with the Security Research team.

Lightbulb ideas are labeled with ~security-research::lightbulb. To help align and communicate the goals of our projects, the ~Security Focus:: and ~Project Goal:: scoped labels are used. One label from each group should be applied to lightbulb issues upon creation. See team labels for details.

Project Prioritization

Projects are prioritized by a mix of factors, including business priorities, project completion stage, and the judgement of the participating security research team member. Project prioritization is a continuous discussion within the team, but will be performed at least once prior to the beginning of each quarter in order to participate in alignment with division OKRs.

The general prioritization order is (from highest to lowest priority):

- Projects in the Product Integration stage.
- Projects in the GitLab-internal Adoption stage.
- Projects aligned with top risk areas as identified by our internal risk assessment.

Once a new project has been prioritized, it will follow the project completion criteria as defined below.

Project Completion Criteria

When a project is considered to be completed depends on the objectives of the project. The objectives for research projects are:

- Idea validation
- GitLab-internal adoption
- GitLab Product integration

All (self-guided) projects start with idea validation and might be expanded to GitLab-internal adoption and/or GitLab Product integration.

Idea Validation

Entry criteria:

- Lightbulb issue
- Alignment with focus area

Exit criteria:

- Research question answered
- Research findings documented and communicated to potential stakeholders

The idea validation phase begins with formulating one or more research questions which we seek to answer with a research project (see also Lightbulb Ideas). The idea should be related to the focus areas of the Security Department to align the work with company objectives. Once the questions are answered, the idea validation phase is completed and the results should be published as a conference talk, blog post, or tech note. The results should also be shared in a focused effort internally with peers in the Security Department and with product managers to which the work might be relevant.

GitLab-internal Adoption

Entry criteria:

- Stakeholder from within the company exist.
- Stakeholder commitment (e.g. willingness to maintain code, operate a service, or triage findings).
- Project plan describing goals and implementation tasks.

Exit criteria:

- Project plan completed

Once an idea has been validated, the research project can be extended to achieve GitLab-internal adoption and/or integration into GitLab's products. If the project is extended, a new

research proposal should be created and buy-in from stakeholders should be obtained. The expected deliverables should be documented. For example, internal adoption can be achieved by documenting research insights in the handbook or by implementing a software service.

Factors to consider when scoping a project for internal adoption or product integration are:

- The time and resources available to the researcher. If the researcher is working on a tight deadline or has limited resources, they may need to scale back the scope of the project or adjust their expectations for completion. In other words, a single person introducing a fundamentally new capability to the product is unrealistic.
- If ongoing maintenance is required, which team will own the maintenance?

Product Integration

Entry criteria:

- Stakeholder from Product exist.
- Stakeholder commitment (e.g. engineering resources for implementation, maintenance, budget).
- Project plan describing goals and implementation tasks.

Exit criteria:

- Project plan completed.

Project Goal	Idea validation	Internal Adoption	Product Integration
Completion Criteria	Research question answered	- Detection capabilities developed - Process improved	- Risk mitigated - Vulnerability fixed
Deliverables	- PoC - Talk, Blog Post, Tech Note	- Code, Infra, Software Service - Handbook MR	- Code, Infra, Software Service - Docs MR
Timeframe (Estimates)	1-2 quarters	2-3 quarters	3-4 quarters

Communicating Results

At the end of each project phase, the team member will share their results. Our goal is to share our results as widely as possible, therefore the most appropriate communication form will be chosen given sensitivity and time constraints. In some cases, due to the sensitivity of the work, the results will only be shared with the company, until which time they can be shared more widely.

Team Labels

Security Focus labels

~Security Focus::
labels are used to align an issue with the broad high-level focus areas of the Security division, based on the risks and priorities of the business. The list

is meant to be stable, but not static.

- Security Focus::Cloud and Infrastructure Security - Related to the secure configuration and use of company production and non-production cloud and infrastructure environments.
- Security Focus::Data Security Governance - Related to the controls, processes, and policies concerning the protection of the data trusted to the company.
- Security Focus::Identity and Access Management - Related to authentication and authorization to business services and data.
- Security Focus::Supply-chain Security - Related to the establishment of trust in 3rd party code, data, and services necessary for the business.
- Security Focus::Other - Related to anything not fitting into the four main focus areas.

Project Goal labels

~Project Goal::
labels are used to communicate the high-level goal of a project.

- Project Goal::Risk Identification & Quantification - The project aims to identify, quantify, and communicate risk(s).
- Project Goal::Risk Mitigation - The project aims to reduce or eliminate a particular risk.
- Project Goal::Team Maturity::Processes - The project aims to mature team processes and improve understanding of how the team works.
- Project Goal::Team Maturity::Technical Growth - The project aims to grow the team's understanding of a technical area. The focus is on learning which can be applied in future projects.

Current and Past Research Projects

Package Hunter

Modern dependency management systems like npm and bundler make reusing code easy and ubiquitous. This increases developer productivity and reduces the time to implement applications. However, dependency management systems also bring new challenges. Even for relatively small applications, the number of 3rd-party dependencies quickly grows to several hundred. Because these dependencies are critical to an application's security, they must be carefully vetted for vulnerabilities and malicious code. Performing this task manually is typically not feasible due to the amount of dependency code being vast and developer time is precious. Automated tools are essential in supporting the review of dependencies and preventing malicious dependencies from entering an application's supply-chain. To tackle this challenge, we have developed Package Hunter, a tool to identify malicious npm modules and Ruby Gems. Package Hunter monitors the system calls of application dependencies during their installation. If any suspicious system calls are observed, an alert is created and brought to the attention of the development team.

Package Hunter is open source. Head over to the project site to try Package Hunter out. There are instructions for running your own Package Hunter instance and the CI template get's you started with analyzing your dependencies in your CI pipeline.

We also welcome contributions. If you are interested in participating in the development of Package Hunter, please see our contribution guide.

The Security Research Team is maintaining a package hunter instance that is used internally at GitLab. The instance is reachable at https://api.package-hunter-live.sec.gitlab.net and https://api.package-hunter.xyz.

GitLab Ecosystem Security Testing

The Security Research team within GitLab conducts security assessments on Open Source Software on a regular basis.

This page describes the reasoning, approach and workflows related to those efforts.

Strengthening the OSS ecosystem for everyone

GitLab relies on a vast amount of Open Source Software, this is not limited to direct code dependencies but also other components in use within the company. To strengthen the overall security posture of GitLab and all other users of our OSS dependencies the Security Research team maintains a list of to-be-assessed OSS projects (internal link). The list is filled from the following categories:

- Cornerstone Project (1)
  - This category is intended for widely used OSS projects with potentially large code bases.
- Dependencies of the GitLab Application (3)
  - Includes code dependencies and other open source projects that ship with GitLab.
- Developer Tooling Projects (3)
  - Projects which are used every day on developer machines.
- Infrastructure/IT Components (3)
  - Projects we rely on while running the company and the product.

In total there are ten projects in four different categories to pick from. The categories are to ensure our work has a broad impact. The projects are chosen and prioritized by the following factors:

- Data access (red/orange/green)
- GitLab API scopes used (if any)
- Functionality provided, especially aiming for high-impact features like:
  - authentication and authorization
  - file access
  - up/download handling
  - handling of secrets
- Adoption within and beyond GitLab, how widely is the project used?

When a project from this list gets assessed the spot on the list will be filled with another project to always keep the funnel filled.

Documentation

Every project and relevant artifacts will be documented internally in the sec-research

repository while the project is ongoing. This repository should be the SSOT for any results and will contain the raw artifacts, write-ups and any PoCs if applicable.

Once the project is concluded and any security issues identified are closed, public facing documentation will be published in the Threat Management tech notes repository. Where applicable, blog posts containing in-depth technical background on the research will be created in collaboration with the External Security Communications team.

Metrics

We're capturing the following metrics for the Ecosystem Security Testing to gain insight into overall progress and impact of the program:

1. Review issues closed per quarter
1. Issues identified per review
1. GitLab improvement issues opened per review

To measure Review issues closed per quarter and Issues identified per review, the following labels were created in the Ecosystem Security issue tracker (internal link):

- ~EcosystemSecurity::Project
- ~EcosystemSecurity::Finding

In order to measure GitLab improvement issues opened per review the following label can be used in the gitlab-com and gitlab-org groups:

- ~SecResearch Followup::Ecosystem

Code Review Sessions

Upon request from team members(internal link) the Security Research team will host "Code Review Sessions"
in which we'll pair on security reviews for a given piece of code. Those sessions will be recorded.

The reviews will all be around vulnerability identification.

The intention for these sessions is learning on both sides, challenge us with some interesting new technologies or software concepts, and in return we'll do our best to share our approaches in vulnerability identification to the given codebase.

Vulnerability Disclosure Guidelines

Vulnerability disclosure can be a delicate process and there is no one-size-fits-all approach for reporting parties. Within the Security Research Team we'll try to report each vulnerability the most effective way, focusing on timely remediation within our GitLab's infrastructure and fix on the vendor's side while respecting embargoes which might be in place.

For third party software listed in our tech stack any vulnerability disclosures should be coordinated with the respective owner of the tech stack item. They might have additional

contacts on the vendor side, or more context how to implement a temporary mitigation for an identified vulnerability.

Depending on the actual risk and exposure it might be needed to further limit the information around the disclosure. In such cases it is recommended to involve the SIRT.

In all cases the team will follow GitLab's Disclosure Guidelines for Vulnerabilities in 3rd Party Software.

Bug bounties and speaker fees

As a result of Security Research it might occur that a team member gets offered a bug bounty for some reported vulnerability which has been identified during their working hours at GitLab. In such a case the payment of the bounty should be instead donated to a charity by the vendor.

The same should be done for potential speaker fees which might be offered for conference presentations which are held on behalf of GitLab.

Any kind of bug bounties/fees being offered based on privately conducted research and speaking engagements are of course not required to be donated to charity.

SECTION: security/product-security/supply-chain-risk-management/_index.md

Introduction

This page outlines GitLab's comprehensive strategy for identifying, assessing, and mitigating risks within our software supply chain. Our risk-based approach is designed to protect both our own data and our customers' data while progressively advancing towards higher levels of Supply chain Levels for Software Artifacts (SLSA) compliance.

Goals and Objectives

- Identify and mitigate risks to GitLab's internal data assets throughout the supply chain
- Safeguard our customers' data by ensuring the integrity and security of our software delivery pipeline
- Achieve and maintain progressively higher levels of SLSA compliance through risk-focused improvements
- Establish comprehensive visibility and traceability to detect supply chain threats and vulnerabilities
- Implement a proactive risk management framework to address emerging supply chain threats
- Create a measurable approach to reduce supply chain attack surfaces
- Support and enhance the Product Security Risk Register (PSRR) with structured supply chain risk data and analysis

Supply Chain Component Model

Decomposing Supply Chain Complexity

GitLab's complete supply chain is too complex to represent in a single comprehensive diagram. Instead, we break down this complexity into manageable components using a recursive model:

- Each artifact has its own supply chain
- Each artifact's supply chain may depend on other artifacts, which have their own supply chains
- This creates a directed graph structure where:
  - Root nodes are the distributed artifacts (our final products)
  - Internal nodes are intermediate artifacts and components
  - Leaf nodes represent the boundaries of our supply chain (external components)

We focus our inventory and management efforts on artifacts and components within GitLab's direct control. For external components (leaf nodes), we track their origin and basic metadata but do not attempt to document their entire supply chains at this stage.

SLSA Supply Chain Model

We align our supply chain tracking with the SLSA framework (Specification 1.1), which defines three key areas to secure:

This model illustrates the core steps we track:

1. Source: Where code is authored, reviewed, and stored
1. Build: Where source is transformed into packages/artifacts
1. Distribution: Where built artifacts are stored and distributed

The model also depicts:

- Producer: The entity responsible for creating the software
- Consumer: The entity consuming the software (another supply chain, or end-user)
- Dependencies: Internal and external components that feed into the Build and Distribution processes

For each artifact in our supply chain, we track its path through these three core steps, documenting controls and provenance at each stage.

Supply Chain component types

Our model identifies specific component types within each core step of the supply chain. These types serve as reference elements to be used when describing a particular subset of the supply chain. Note that not all components will be present in every supply chain - the categorization below provides a framework for comprehensive modeling.

These types are linked to SLSA threats in the PSRR to create comprehensive risks, see the "Threats" section below. Risks can be linked to a subtype if more granularity is needed.

Source components

The Source core step includes everything that can edit and alter the source code before the Build step:

Component type	Sub type	Label
Development Dependencies	(Wraps all sub-types below)	~sscs-rm-component:src:dev-dependencies
	Development environment setup tools and dependencies (ex: asdf/mise)	~sscs-rm-component:src:dev-setup-tools
	IDEs (including extensions and plugins)	~sscs-rm-component:src:IDEs
	Docker images	~sscs-rm-component:src:docker-images
	Pre-commit hooks	~sscs-rm-component:src:pre-commit-hooks
	Local code formatters and linters	~sscs-rm-component:src:linters
GitLab Repositories	(Wraps all sub-types below)	~sscs-rm-component:src:repo
	Project configuration	~sscs-rm-component:src:repo-config
	Code Owners configuration	~sscs-rm-component:src:repo-code-owners
	Repository access controls	~sscs-rm-component:src:repo-access-control

Build components

The Build core step includes everything that can transform the source code (compilation, linting, etc.) and produce an artifact:

Component type	Sub type	Label
CI/CD	(Wraps all sub-types below)	~sscs-rm-component:build:ci-cd
	GitLab Runners	~sscs-rm-component:build:gitlab-runners
	CI/CD templates	~sscs-rm-component:build:ci-templates
	CI/CD Components	~sscs-rm-component:build:ci-components
Build images	(Wraps all sub-types below)	~sscs-rm-component:build:build-images
	Base Docker images	~sscs-rm-component:build:base-docker-images
	Intermediate images	~sscs-rm-component:build:intermediate-images
	Container build tools	~sscs-rm-component:build:container-build-tools
	Container registry	~sscs-rm-component:build:container-registry
Runtime Dependencies	(Wraps all sub-types below)	~sscs-rm-component:build:runtime-dependencies
	Ruby Gems	~sscs-rm-component:build:ruby-gems
	NPM packages	~sscs-rm-component:build:npm-packages
	Go modules	~sscs-rm-component:build:go-modules
	Python packages	~sscs-rm-component:build:python-packages
	Other language-specific dependencies	~sscs-rm-component:build:other-packages
Secrets	(Wraps all sub-types below)	~sscs-rm-component:build:secrets
	Vault	~sscs-rm-component:build:vault
	CI/CD variables	~sscs-rm-component:build:ci-variables
	Key management systems	~sscs-rm-component:build:key-management
	Certificate authorities	~sscs-rm-component:build:certificate-authorities
	Signing infrastructure	~sscs-rm-component:build:signing-infra

Distribution components

Component type	Sub type	Label
Registries	(Wraps all sub-types below)	~sscs-rm-component:dis:registries
	Package registry	~sscs-rm-component:dis:package-registry
	Container registry	~sscs-rm-component:dis:container-registry
Distribution Infrastructure	(Wraps all sub-types below)	~sscs-rm-component:dis:distribution-infra
	CDNs	~sscs-rm-component:dis:cdns
	Mirror services	~sscs-rm-component:dis:mirror-services
	Download servers	~sscs-rm-component:dis:download-servers
Verification Systems	(Wraps all sub-types below)	~sscs-rm-component:dis:verification-systems
	Signature verification	~sscs-rm-component:dis:signature-verification
	Checksumming services	~sscs-rm-component:dis:checksumming-services
	Attestation systems	~sscs-rm-component:dis:attestation-systems

SLSA 1.1 Alignment

This framework is based on the Supply chain Levels for Software Artifacts (SLSA) specification 1.1. We deliberately adopt SLSA terminology and concepts to ensure consistency with industry standards and facilitate compliance efforts. Key SLSA elements incorporated into our model include:

- Build provenance documentation
- Source verification
- Build integrity controls
- Artifact authentication
- Access control requirements

Threats

SLSA defines a set of threats that are used in the PSRR to link elements of the model to risks:

Threat area	Threat	Description	Label
Source	(A) Producer	Software producer intentionally creates a malicious revision of the source	~sscs-rm-threat::a-malicious-source
	(B) Modifying the source -> (B1) Submit change without review	Directly submit without review	~sscs-rm-threat::b1-submit-without-review
		Single actor controls multiple accounts	~sscs-rm-threat::b1-actor-controls-multiple-accounts
		Use a robot account to submit change	~sscs-rm-threat::b1-robot-account-submit
		Abuse of rule exceptions	~sscs-rm-threat::b1-abuse-rule-exceptions
		Highly-permissioned actor bypasses or disables controls	~sscs-rm-threat::b1-bypass-controls
	(B) Modifying the source -> (B2) Evade change management process	Modify code after review

| ~sscs-rm-threat::b2-modify-after-review |
		Submit a change that is unreviewable	~sscs-rm-threat::b2-unreviewable-change
		Copy a reviewed change to another context	~sscs-rm-threat::b2-copy-to-another-context
		Commit graph attacks	~sscs-rm-threat::b2-commit-graph-attacks
	(B) Modifying the source -> (B3) Render code review ineffective	Collude with another trusted person	~sscs-rm-threat::b3-collusion
		Trick reviewer into approving bad code	~sscs-rm-threat::b3-trick-reviewer
		Reviewer blindly approves changes	~sscs-rm-threat::b3-blind-approval
	(B) Modifying the source -> (B4) Render change metadata ineffective	Forge change metadata	~sscs-rm-threat::b4-forge-metadata
	(C) Source code management	Platform admin abuses privileges	~sscs-rm-threat::c-admin-abuse
		Exploit vulnerability in SCM	~sscs-rm-threat::c-exploit-scm-vulnerability
Build	(D) External build parameters	Build from unofficial fork of code	~sscs-rm-threat::d-unofficial-fork
		Build from unofficial branch or tag	~sscs-rm-threat::d-unofficial-branch
		Build from unofficial build steps	~sscs-rm-threat::d-unofficial-steps
		Build from unofficial parameters	~sscs-rm-threat::d-unofficial-parameters
		Build from modified version of code modified after checkout	~sscs-rm-threat::d-modified-after-checkout
	(E) Build process	Forge values of the provenance (other than output digest)	~sscs-rm-threat::e-forge-provenance-values
		Forge output digest of the provenance	~sscs-rm-threat::e-forge-output-digest
		Compromise project owner	~sscs-rm-threat::e-compromise-owner
		Compromise other build	~sscs-rm-threat::e-compromise-other-build
		Steal cryptographic secrets	~sscs-rm-threat::e-steal-secrets
		Poison the build cache	~sscs-rm-threat::e-poison-cache
		Compromise build platform admin	~sscs-rm-threat::e-compromise-platform-admin
	(F) Artifact publication	Build with untrusted CI/CD	~sscs-rm-threat::f-untrusted-cicd
		Upload package without provenance	~sscs-rm-threat::f-upload-without-provenance
		Tamper with artifact after CI/CD	~sscs-rm-threat::f-tamper-after-cicd
		Tamper with provenance	~sscs-rm-threat::f-tamper-with-provenance
	(G) Distribution channel	Build with untrusted CI/CD	~sscs-rm-threat::g-untrusted-cicd
		Issue VSA from untrusted intermediary	~sscs-rm-threat::g-untrusted-vsa
		Upload package without provenance or VSA	~sscs-rm-threat::g-upload-without-verification
		Replace package and VSA with another	~sscs-rm-threat::g-replace-package-vsa
		Tamper with artifact after upload	~sscs-rm-threat::g-tamper-after-upload
		Tamper with provenance or VSA	~sscs-rm-threat::g-tamper-verification
Usage	(H) Package selection	Dependency confusion	~sscs-rm-threat::h-dependency-confusion
		Typosquatting	~sscs-rm-threat::h-typosquatting
	(I) Usage	Improper usage	~sscs-rm-threat::i-improper-usage
Dependency	Build dependency	Include a vulnerable dependency	~sscs-rm-threat::dep-vulnerable-dependency
		Use a compromised build tool	~sscs-rm-threat::dep-compromised-build-tool
		Use a compromised runtime dependency during the build	~sscs-rm-threat::dep-compromised-runtime-dependency

Integration with the Product Security Risk Register

The Supply Chain Risk Management Strategy serves as a critical foundation for the Product Security Risk Register (PSRR). Each supply chain-related risk identified in the PSRR must be linked to specific elements within this supply chain model:

1. Risk Mapping Requirements

   - Every supply chain risk in the PSRR must reference specific components from this model
   - By extension, risks identifies which part of the supply chain step is affected (Source, Build, or Distribution)
   - Risk documentation can include specific artifacts involved
   - The potential for risk propagation through the supply chain should be documented

1. Bidirectional Traceability

   - Supply chain model components must link back to relevant PSRR risk items (see labels above)
   - PSRR entries must link to the affected supply chain components
   - Updates to the supply chain model should trigger reviews of related PSRR entries
   - New PSRR risks related to supply chain must be mapped to this model during risk registration

1. Unified Risk Assessment Approach

   - The risk scoring methodology must be consistent between this model and the PSRR
   - Supply chain risk mitigations documented in the PSRR should align with controls in this model
   - Risk acceptance decisions should consider the full context of the supply chain graph

This integration ensures a comprehensive approach to supply chain risk management that leverages our existing security frameworks while providing deeper visibility into supply chain-specific threats.

How to Use This Model

For Development Teams

{{% alert title="Note" color="primary" %}}
The following items represent future/North Star requests and are not current requirements.
{{% /alert %}}

1. Artifact Documentation

   - For each new artifact type, document its source components
   - Register artifact in the central inventory (see https://gitlab.com/groups/gitlab-org/-/epics/16484)

1. Compliance Verification

   - Regular self-assessments against the model requirements
   - Document evidence of control implementation

- Participate in periodic supply chain reviews

For Security Teams

1. Risk Monitoring and Intelligence

- Implement continuous risk monitoring for registered components
- Conduct regular threat hunting across the supply chain ecosystem
- Maintain a supply chain threat intelligence program
- Establish risk thresholds and escalation procedures for detected anomalies

1. Risk-Based Audit Support

- Maintain risk evidence collection for compliance purposes
- Support external audits with risk assessment documentation
- Verify the implementation and effectiveness of risk controls across teams
- Develop risk-focused audit narratives and documentation

1. Risk Model Evolution

- Update the risk model as new threats and attack techniques emerge
- Refine risk classification criteria based on incident data and operational feedback
- Ensure alignment with evolving SLSA requirements and industry threat landscape
- Conduct periodic red team exercises to test supply chain security resilience

Risk Management Metrics and Success Criteria

{{% alert title="Note" color="primary" %}}
The following metrics represent future/North Star indicators. These are not currently tracked and are subject to changes.
{{% /alert %}}

Success in our supply chain risk management strategy will be measured by:

Metric	Possible methodology	Dependencies
Completeness of risk assessment coverage across all components and artifacts	Track threat models done for each component.	Inventory of GitLab public artifacts
Quantifiable reduction in supply chain security incidents and vulnerabilities	Create new labels to track down incidents and vulnerabilities related to our supply chain.	AppSec team
Decreased mean time to detect and respond to supply chain threats	Risks in the PSRR should have remediation issues linked, but also detection issues.	PSRR
Progressive achievement of higher SLSA levels with documented risk reduction	Track implemented SLSA requirements.	This Epic for SLSA Level 3 support.
Successful passing of external security audits with minimal findings	Map findings related to supply chain. Loop back with coverage above to make sure previously unknown risks are logged.	SecAssurance / AppSec
Improved visibility and quantification of supply chain risks and dependencies	Track "dead-ends" in supply chains (missing information).	Each risk is labeled correctly in the PSRR & Inventory of GitLab public artifacts

| Reduced number of critical and high-risk components in the supply chain | Number of components with risk score above a shreshold. | PSRR |

SECTION: security/product-security/vulnerability-management/_index.md

Vulnerability Management is the continual process of identifying, prioritizing, mitigating and remediating vulnerabilities. At GitLab we identify vulnerabilities in a number of different ways depending on the component being analyzed. This process and associated tooling is owned by the Vulnerability Management team.

This page primarily outlines our vulnerability management policies and procedures at GitLab. The policies and procedures used to manage vulnerabilities at GitLab are collectively referred to as the Vulnerability Management Standard.

Quick reference

- Runbook: I have a vulnerability finding, what should I do?
- Runbook: How are vulnerabilities commonly fixed in development?
- I have a vulnerability detection which I can't fix within SLA (or at all), what should I do?
- What is a vulnerability?
- Why should we fix vulnerabilities?
- When is a vulnerability considered "fixed"?
- When can I close a vulnerability tracking issue?
- Who is Vulnerability Management at GitLab, and what do they do?
- Which automation tools run GitLab's vulnerability management workflow?

What does Vulnerability Management cover at GitLab?

Vulnerability Management covers all systems which store, process or transmit GitLab production and/or GitLab customer data, as well as GitLab-managed software and software dependencies, based on the GitLab critical system tiering methodology. Specifically:

- GitLab-managed infrastructure
- GitLab software, packages, images & dependencies
- Detected Later (HackerOne reports, Community or Team Member reported bugs, PenTest findings)

Approach & Strategy

Vulnerability Management at GitLab focuses primarily on gathering data about vulnerabilities, vulnerability detection and attack surface. We do this by building tooling to gather, normalize and correlate various sources of data into actionable automated workflows which either directly address or mitigate findings. If this cannot be done automatically, we aim to make the output of this process easily available and understandable to the GitLab DRIs responsible for mitigating or fixing a finding. Our goal is to feed this tooling back into GitLab itself, and where it makes sense, to build and use GitLab features to support our own and our customer's workflows wherever possible.

Vulnerability Management Standard

- Where possible, GitLab should be used as the source of truth for vulnerability detection and management. GitLab issues are currently used to track vulnerability management activities in addition to engineering work required to remediate vulnerabilities.
- Application (WebApp and container) vulnerability scanning must occur on a minimum of a monthly basis, and use scanners and scanner configuration tuned for the code being scanned. This includes enabling appropriate dependency scanning, container scanning, secrets scanning, and SAST scan jobs. See the Secure your application documentation for all available scanners. Where possible, standard supported GitLab scanners should be used in CI based on supported templates or components.
- Container images shipped by GitLab should be scanned at a minimum with GitLab container scanning before being shipped.
- Infrastructure (operating systems and databases) vulnerability scanning must occur on a minimum of a monthly basis.
- Authenticated/credentialed scans from a privileged network domain, in addition to external scanning of infrastructure, should be performed wherever possible.
- Vulnerability scanners must be up to date (the latest operable version), and hardened to resist unauthorized use or modification.
- Third party penetration testing must be performed annually.
- Even if mitigated or not exploitable, vulnerable code (including dependencies) must be removed or updated to non-vulnerable versions to resolve vulnerability findings, so we are not shipping vulnerable code. For more information on why we do this, please see Why should we fix vulnerabilities?
- Vulnerabilities must be fixed within SLA timeframes, unless an exception exists. Where vulnerabilities are not fixed within SLA, the responsible group should work to understand which factors contributed to missing remediation SLA timeframes as part of regular group development retrospectives.

Service Level Agreements (SLAs) / Service Level Objectives (SLOs)

For information on vulnerability management SLAs / SLOs, see the Vulnerability Management SLAs handbook page.

Roles & Responsibilities

Role	Responsibility	Notes
Vulnerability Management	Responsible for implementing and maintaining Vulnerability Management standards. Develop and maintain (VulnMapper). Evaluate severity and priority, analyze the actual impacts, and make risk adjustment for the vulnerabilities which VulnMapper is unable to process automatically. Iterate on data gathering, normalization and correlation automation and work to provide actionable insights on mitigation and remediation to GitLab team members.	
Security Compliance Team	Responsible for oversight of supporting procedures as part of ongoing continuous control monitoring for adherence with compliance baselines. Responsible for approving Deviation Requests (or getting approval from Agency) and closing DR issues once all linked vulnerability issues have been closed.	
Application Security Team	Collaborate with Development & Engineering in the triage of vulnerabilities and provide stable counterparts TODO.	
Development & Engineering	Responsible for mitigating and remediating vulnerabilities in GitLab-managed software within SLA	

Reliability	Responsible for mitigating and remediating vulnerabilities in GitLab-managed infrastructure within SLA	
Product Security Engineering Team	Develop and maintain the GitLab Security Bot, a.k.a. appsec-escalator, which drives SLA and remediation reminders	AppSec collaborates with Product Security Engineering and Vulnerability Management in maintaining this bot.
Infrastructure Security	Responsible for maintaining cloud configuration and resolving vulnerabilities or misconfiguration in cloud policy and other security configuration	
Security Leadership (Code Owners)	Responsible for approving changes to this standard	

Additional Notes for Vulnerability Triage

1. VulnMapper creates vulnerability tracking issues and if not available already, adds appropriate severity, priority and other necessary labels. Vulnmapper is currently opt-in for issue creation, if issues are not created for your group, please let Vulnerability Management know.
1. For the vulnerabilities which VulnMapper can process automatically, Vulnerability Management updates vulnerability tracking issues with the necessary information such as CVSS score, severity and priority, package name and version etc. Current capabilities are documented on the automation handbook page.
1. For supported OS bases, VulnMapper also adds labels indicating fix availability to further reduce triage time, based on vendor advisory and package information. Supported OSes for this functionality is limited, and is detailed in the automation handbook page.
1. Where fixes are not available or differences between NVD and vendor severity exists, VulnMapper reports Deviation Request issues for vendor dependencies and vendor risk adjustments automatically.
1. Vulnerability Management assigns vulnerability issues to a development group based on available information. VulnMapper also is configured with a map of ownership information for GitLab development groups, and will appropriately label issues for different groups with the appropriate triage group labels, when updating or creating issues.
   1. Where possible, Vulnerability Management, via our automation, where available, will make recommendations on how to address vulnerability detections, either via an MR or the description in vulnerability tracking issues.
   1. Vulnerability triage is a shared responsibility although Vulnerability Management is the DRI. AppSec, InfraSec and respective feature domain development groups are responsible for providing evidence and/or analysis of GitLab-specific impact during vulnerability triage upon Vulnerability Management's request. Vulnerability tracking issues should be assigned to the appropriate AppSec stable counterpart or development group for further analysis and after this, for remediation, the responsible development or infrastructure group becomes the owner and DRI, as they will have the context and responsibility over impacted vulnerable assets.
1. Vulnerability is now ready for remediation by the responsible group

Contact

If you have any questions or concerns related to vulnerability management please contact Vulnerability Management and Engineering in the gsecurityvulnmgmt or the security channels on Slack, you can also open an issue in the Vulnerability Management issue tracker. All work being done to improve this process is also tracked in the issue tracker, and we'd love your feedback.

Exceptions

If a you are mitigating or addressing a vulnerability, and you do not believe the GitLab Vulnerability Management standards can be met, please raise an exception request. For details on exception requests, handling, including raising a request for an SLA exception, see the SLA Exception handbook page.

References

- Application Vulnerability Management Procedure
- Bug Bounties

SECTION: security/product-security/vulnerability-management/automation.md

Vulnerability management maintains automation tooling which intends to automate as much of this standard and the related procedures as possible. We're always iterating on ways we can save time for all stakeholders whilst keeping GitLab and our users safe.

VulnMapper

VulnMapper is the main automation tool used by Vulnerability Management. It is a closed-source tool with a goal of automating security processes and building functionality into GitLab as part of Product Security's dogfooding goals.

VulnMapper Features

The key features of VulnMapper are:

- Ingestion of vulnerabilities from infrastructure, code and configuration security finding sources
- Ingestion of vulnerability metadata from advisory sources, including GLAD
- Normalization and correlation of vulnerability advisories and findings
- Creation of tracking issues in GitLab focused on resolving vulnerabilities
- Automated handling of common SLA exception scenarios

Known limitations

Current issues & limitations are tracked on the private VulnMapper issue tracker. Some of the key limitations currently are:

- Container fix information is not as reliable as package-based finding information, and as a result package-based fix availability labels are considered more trustworthy
- No advisory and fix availability information is available for OSes other than RHEL/UBI or for dependency/language package findings

Supported OSes

Whilst vulnmapper aims to support all findings, determining the availability of fixes for packages associated with vulnerabilities relies on us having good quality data sources for advisories and related fix availability.

The current list of base OSes and therefore base images we support for fix availability tracking in vulnmapper are:

- Red Hat Enterprise Linux 7, 8, 9
- Red Hat Universal Base Image (UBI) 7, 8, 9

Only findings for these supported OSes will have package fix determined, and therefore labelled on related GitLab vulnerability tracking issues.

We are working on adding Debian and Alpine fix availability currently, and welcome team members filing issues for other base OS package types they would like to see supported.

For any feedback or ideas for improvement of the vulnerability automation used at GitLab, team members can file an issue. Additionally, if GitLab team members want to onboard their images and environments or talk about vulnerability automation, please reach out via Slack (in gsecurityvulnmgmt) or create an issue in our (private) tracker here.

Providers

VulnMapper uses a provider model to integrate with external systems, including GitLab. Providers are responsible for populating or acting on the VulnMapper database, including tasks like ingesting advisory information (from sources like NVD, Red Hat, Ubuntu), vulnerability detections (from sources like GitLab scans, Wiz, AWS Inspector), and automatically creating or updating tracking issues in tracking providers (currently using GitLab issues).

Issue Creation

Currently at GitLab, certain groups have opted in to have VulnMapper automatically handle triage & creation of tracking issues for vulnerability findings in assets they ship. Currently this includes:

- Distribution (including CNG images)
- Runner

For these groups, VulnMapper will automatically execute scans of relevant container images using GitLab, and after normalization and correlation of relevant advisory information, created tracking issues in GitLab correctly labelled for remediation. Team members performing triage and remediation work on created issues should refer to the standard vulnerability management labels page as between the applied labels and the description of the issue, whether or not the issue is actionable should be clear. Typically, these issues will require updating of a package or base image in order to resolve the findings, if an updated package or image is available. If it is not, the issue should be labelled as such and can have the priority lowered.

Issue Closure

Under the following circumstances VulnMapper will also automatically close tracking issues to improve team efficiency.

- When a fix will not be made available by the vendor of a dependency, and a deviation request has been opened automatically

- When the linked vulnerability finding in GitLab or another supported VulnMapper provider is resolved or dismissed, when issue closure is enabled for the project

Issue closure is currently enabled for the following groups:

- Distribution (including CNG images)
- Production Infrastructure findings (as detected by Wiz)
- Dedicated Infrastructure findings (as detected by AWS Inspector)

Deviation Request Automation

As part of GitLab's FedRAMP compliance, and the SLA exception process, operational requirement and risk adjustment Deviation Requests are created automatically by VulnMapper when datasources used by VulnMapper indicate there is a difference between the baseline (NVD) vulnerability impact and the impact (severity) as assessed by the vendor of the component (risk adjustment) or when assessment by a vendor indicates that a fix will not be made available (operational requirement). In these cases a Deviation Request issue will be populated and created automatically, and in the case of fixes not being available, the vulnerability tracking issue will be labelled and linked to this deviation request and then closed, as the tracking issue is not actionable by team members. No team member interaction is required for this automated process to take place.

SECTION: security/product-security/vulnerability-management/closing-issues.md

At GitLab, we use GitLab to track the detection and remediation of vulnerability findings. In the case of remediation work, a vulnerability tracking issue (which is a regular GitLab issue) is used to track work required to remediate a specific vulnerability finding, typically against one or more assets where the vulnerability was detected.

In order to close a vulnerability tracking issue, one of the following criteria must be met:

1. The finding has been remediated and validated as remediated, and made available to intended users
1. The finding was a false positive detection, and required work has been performed to prevent re-detection
1. A finding was in a third-party dependency from a vendor, and the vendor has indicated they will not fix the vulnerability due to lack of impact, and an SLA exception or Deviation Request has been raised

In other situations, there is typically more work which is required before the issue can be closed, to ensure the detection is not erroneously closed and the finding forgotten about, and issue closure is not appropriate as a result.

Further guidance on mitigating and resolving vulnerabilities is available in the following runbooks:

- So you have a vulnerability finding?
- Fixing vulnerabilities

If you require further guidance, for vulnerabilities in GitLab code, you can reach out to your AppSec stable counterpart, or to Vulnerability Management.

SECTION: security/product-security/vulnerability-management/development-and-code-review.md

Purpose

This standard documents requirements for all engineering & development work which happens in the Vulnerability Management team.

Scope

This policy sets requirements for:

- Documenting issues and features which require engineering or development work
- General guidelines around code contributions and IDE configuration
- Sizing, scoping and planning engineering work for inclusion in a milestone during milestone planning
- Estimating and communicating the weight (estimated time) size (based on weight) of engineering or development work
- Reviewing the code of team members when contributing to Vulnerability Management tools
- How we decide to include something in a milestone and schedule work

This policy is only applicable to projects owned and maintained by the Vulnerability Management team, which are used as part of delivering, monitoring and automating processes required to maintain the Vulnerability Management Standard for GitLab. It does not cover contributions the team makes to other projects, including the GitLab product itself and associated projects.

Roles & Responsibilities

Responsibility	Role(s)
Final say over overall milestone planning and capacity management	Engineering Manager
Maintaining this standard, and reviewing feedback and suggestions to this standard	Staff Security Engineer / Engineering Manager
Documenting, scoping, sizing and labeling engineering work they are planning per this standard	Security Engineer
Contributing code as merge requests for review following guidelines set in this standard	Security Engineer
Reviewing the contributions of other team members to Vulnerability Management projects per this standard	Security Engineer

Definitions

Code Review

Code review is the process and rules by which one team member is expected to assist another team member in validating that a merge request containing code meets not only this standard,

but the stated goal of the MR and related issues. This standard applies to what must and what must not be done when reviewing the work of other team members for merging into new or existing tools and applications used by the Vulnerability Management team.

Development

Development refers to making changes to or building new programs and applications using programming languages and related configuration to achieve a documented outcome.

Engineering

Engineering work includes development work, but also covers related work such as infrastructure, planning and any related documentation work. Primarily this standard applies to engineering work, as the majority of the processes, external systems and tools we use are managed by code, or are code.

Priority

Priority is a measure of how an issue aligns with the overall priorities of the Vulnerability Management team. The Vulnerabi